COMMON COURSE ASSIGNMENT

CSA08 – GENERATIVE AI & PROMPT ENGINEERING | Covering Large Language Models & Prompt-Driven Engineering Assistance

A. Assignment Information

Department	Computer Science and Engineering
Programme	B.E. / B.Tech – CSE
Course Code & Course Name	CSA08 – Generative AI and Prompt Engineering
Academic Year / Batch	2026 – 2027
Faculty Name	Dr.Chandravathi C
Assignment Title	Design and Implementation of PromptCraft – A Prompt-Driven Engineering Assistant using Large Language Models and Prompt Engineering
Date of Issue	21.08.2026
Date of Submission	03.09.2026
Maximum Marks	100
Course Outcome(s) – CO	CO1 & CO2
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyze, L5 – Evaluate
SDG Mapping (SDG 1–17)	SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Prompt-driven AI assistants are increasingly used across the software industry to accelerate coding, debugging, documentation, and requirement analysis, making skills in large language models and prompt engineering directly relevant to modern engineering practice.

B. Assignment Problem / Challenge

This assignment requires students to design, implement, and evaluate an AI-powered engineering assistant that applies core Generative AI concepts — prompt design and engineering techniques (zero-shot, few-shot, and chain-of-thought prompting), large language model (LLM) API integration, and structured evaluation of model outputs — to a unified engineering-support platform called **Prompt Craft**. Students must integrate multiple prompting strategies, build task-specific engineering modules, deploy the system as a lightweight web application, validate output quality and reliability with sample test cases, and address basic responsible-AI considerations such as transparency of prompts, hallucination handling, and reproducibility of results.

C. Problem Statement

Scenario: "PromptCraft" – A Prompt-Driven Engineering Assistant

Many students and practicing engineers find it difficult to consistently get high-quality, reliable outputs from large language models without structured prompting. Your task is to design and implement an interactive engineering-support web application called **PromptCraft** that helps engineers construct effective prompts, apply prompt engineering techniques, and use an LLM to accelerate common software-engineering tasks through a unified interface.

The system shall integrate three major functional modules:

1. Prompt Engineering Workbench

Implement zero-shot, few-shot, and chain-of-thought prompting templates for common engineering tasks.

- Users should be able to enter a task description, select a prompting strategy, and optionally supply few-shot examples or context.
- The system must display the fully constructed prompt sent to the LLM together with the generated response, enabling students to see exactly how prompt structure affects output quality.

2. LLM-Powered Engineering Task Modules

Implement at least three engineering-assistant task modules, e.g., code generation & explanation, automated debugging assistance, and requirement-to-documentation generation.

- Users will enter their engineering artifact (a code snippet, an error/bug description, or a requirement statement) as input to the selected module.

- The system should show the underlying prompt template used for each module, the LLM output, and any post-processing applied, so that students can trace how the assistant arrived at its result.

3. Prompt Strategy Evaluation & Comparison

Implement comparison of at least three prompting strategies (e.g., zero-shot, few-shot, chain-of-thought) for the same task.

- Users will enter a representative engineering task and a set of reference/expected outputs for evaluation.
- The system should display response quality, response time / latency, and token usage for each strategy, allowing clear comparative analysis.

Design the application workflow so that user inputs (task descriptions, code snippets, bug reports, requirement statements) are processed through the corresponding prompt templates and LLM calls, and the results are presented as clear prompts, responses, and comparative metrics. Develop a clean, simple web interface using **Streamlit**. Provide 2–3 sample test cases for each module to demonstrate effectiveness. Students must use a real or simulated LLM API (e.g., OpenAI, Anthropic, or an open-source model) to generate responses.

Evaluate the application under suitable test cases and analyze the quality of LLM outputs, usability of the interface, and clarity of the prompt/response traces. The completed system should also briefly address responsible-AI considerations including transparency of prompts, hallucination and error handling, data privacy, and reproducibility of results.

D. Requirements and Constraints

- **Functional:** Provide interactive modules for a Prompt Engineering Workbench (with visible prompt construction and response display), LLM-Powered Engineering Task Modules (code generation, debugging assistance, and documentation generation), and Prompt Strategy Evaluation (quality, latency, and token-usage comparison). Accept user inputs as well as sample data. Produce comparative tables and prompt/response traces.
- **Interface:** Clean, simple, and user-friendly Streamlit web application with separate pages or clearly separated sections for each module.
- **Performance:** LLM calls must return results with acceptable latency for interactive classroom use and support clear comparative analysis across prompting strategies.
- **Technical constraint:** Use Python + Streamlit. No database or complex architecture required. Use any accessible LLM API or open-source model for prompt execution.

- **Reliability:** Prompt templates and task modules must behave consistently and reproducibly; intermediate prompts must be transparent; the system must handle malformed inputs and LLM API errors without crashing.
- **Resource constraint:** Project must be completable within the semester timeline using standard student computing resources.
- **Responsible Computing:** Ensure transparency of prompts and model limitations, reproducibility of results, mitigation of hallucinated or misleading outputs, and adherence to responsible-AI and data-privacy practices.
- **Evaluation & Submission:** Test all three modules with sample inputs and show representative results. Submit working Streamlit code, a short report containing sample prompts/outputs / screenshots, comparative analysis, and a GitHub repository link.

E.Student Work

1. Problem Understanding and Formulation

PromptCraft is an interactive engineering-support application that improves the quality and consistency of LLM responses through structured prompt engineering. The problem is to provide one interface where a user can select a prompting strategy, view the exact constructed prompt, execute an LLM/simulator, and evaluate the resulting response.

Inputs include an engineering task, optional examples/context, source code, bug descriptions, or requirements. Outputs include the constructed prompt, generated response, quality assessment, latency, and estimated token usage.

Assumptions: classroom-scale usage, non-sensitive test data, Python/Streamlit execution, and either a real or simulated LLM. The design must satisfy Section D: modularity, transparency, acceptable latency, reproducibility, error handling, and responsible-AI practices.

2. Objective and Expected Outcomes

The objective is to design and implement PromptCraft using zero-shot, few-shot, and chain-of-thought-style prompting for practical software-engineering tasks.

Expected outcomes: a working Streamlit interface; three engineering modules; visible prompt/response traces; strategy comparison; measurable quality, latency, and token metrics; graceful input/API error handling; and documented responsible-AI considerations.

3. Requirements, Constraints and Assumptions

Functional requirements: strategy selection, task entry, optional examples, prompt construction, response generation, code generation/explanation, debugging assistance, requirement-to-documentation, and comparative evaluation.

Technical constraints: Python + Streamlit, no database or complex architecture, standard student computing resources, and a real or simulated LLM.

Reliability and ethics: validate inputs and outputs, do not hard-code secrets, avoid unnecessary sensitive data, expose prompt structure, and clearly state model limitations.

4. Application of Relevant Course Knowledge / Concepts

Prompt engineering is applied through zero-shot prompting for direct instructions, few-shot prompting for example-guided output, and chain-of-thought-style instructions for systematic problem analysis. LLM integration is separated from prompt construction so the simulator can be replaced by an approved API.

Evaluation applies controlled comparison: the same engineering task is executed with different strategies and compared using quality, latency, and estimated token usage. Software-engineering concepts are applied through modular functions, reusable templates, input validation, test cases, and error handling.

5. Solution / Design / Methodology

The proposed architecture is: User Interface → Prompt Builder → LLM/Simulator → Output Validation → Metrics → Results Dashboard.

Workflow: accept task → select strategy → construct transparent prompt → execute model → validate response → calculate metrics → display result. For comparison, repeat the workflow for all three strategies and display the results in one table.

Three task modules are implemented: Code Generation & Explanation, Automated Debugging Assistance, and Requirement-to-Documentation. The design is intentionally lightweight to satisfy Section D.

6. Algorithm / Pseudocode / Flowchart

Algorithm: Prompt Strategy Evaluation

1. Start.
2. Read the engineering task and optional examples.

3. Select Zero-Shot, Few-Shot, or Chain-of-Thought.

4. Build the corresponding prompt template.

5. Send the prompt to the LLM/simulator.

6. Record response, latency, and estimated token usage.

7. Evaluate response quality.

8. Repeat steps 3–7 for the remaining strategies.

9. Display comparative results.

10. Select the strategy with the best quality–latency–token trade-off.

11. Stop.

Flowchart: Start → Input Task → Select Strategy → Build Prompt → LLM → Validate → Metrics → Display → Compare → End.

7. Implementation / Source Code

The following implementation is a compact working Streamlit prototype. Save it as app.py and run it using: streamlit run app.py.

```
import streamlit as st
import time

st.set_page_config(page_title="PromptCraft", layout="wide")
st.title("PromptCraft - Engineering Assistant")

def build_prompt(task, strategy, examples=""):
    if strategy == "Zero-Shot":
        return f"You are an engineering assistant.\nTask: {task}\nGive a clear solution."
    if strategy == "Few-Shot":
        return f"You are an engineering assistant.\nExamples:\n{examples}\nTask: {task}\nFollow the pattern."
    return f"You are an engineering assistant.\nAnalyze systematically and check edge cases.\nTask: {task}"

def llm(prompt):
    # Replace with an approved LLM API for real deployment.
    p = prompt.lower()
    if "debug" in p or "error" in p:
```

```
        return "Debug: locate the fault, check inputs/boundaries, fix the cause, and retest."
    if "document" in p or "requirement" in p:
        return "Documentation: purpose, inputs, process, outputs, constraints and tests."
    return "Engineering response: requirements, design, implementation, edge cases and testing."

def run(task, strategy, examples=""):
    prompt = build_prompt(task, strategy, examples)
    start = time.perf_counter()
    response = llm(prompt)
    latency = time.perf_counter() - start
    tokens = len((prompt + response).split())
    quality = min(100, 70 + len(response)//20)
    return prompt, response, latency, tokens, quality

strategy = st.sidebar.selectbox(
    "Prompt Strategy", ["Zero-Shot", "Few-Shot", "Chain-of-Thought"]
)
task = st.text_area("Engineering Task",
                    "Generate Python code to validate an email address.")

examples = ""
if strategy == "Few-Shot":
    examples = st.text_area("Few-Shot Examples",
                            "Input: phone\nOutput: validate length and digits.")

if st.button("Generate Response"):
    if not task.strip():
        st.warning("Enter a task.")
    else:
        prompt, response, latency, tokens, quality = run(task, strategy, examples)
        st.subheader("Constructed Prompt")
```

```
        st.code(prompt)
        st.subheader("LLM Response")
        st.write(response)
        st.write(f"Quality: {quality}/100 | Latency: {latency:.4f}s | Tokens: {tokens}")

st.divider()
st.header("Engineering Task Modules")
module = st.selectbox("Module", [
    "Code Generation & Explanation",
    "Automated Debugging Assistance",
    "Requirement-to-Documentation"
])
artifact = st.text_area("Code / Bug / Requirement")

if st.button("Run Module"):
    if not artifact.strip():
        st.warning("Enter an artifact.")
    else:
        if module == "Code Generation & Explanation":
            t = "Generate and explain code for: " + artifact
        elif module == "Automated Debugging Assistance":
            t = "Debug this problem and explain the correction: " + artifact
        else:
            t = "Convert this requirement into documentation: " + artifact
        prompt, response, latency, tokens, quality = run(t, "Zero-Shot")
        st.subheader("Module Prompt")
        st.code(prompt)
        st.subheader("Module Output")
        st.write(response)

st.divider()
```

```
st.header("Prompt Strategy Evaluation")
compare_task = st.text_area(
    "Comparison Task", "Explain how to optimize a slow Python function."
)

if st.button("Compare Strategies"):
    rows = []
    for s in ["Zero-Shot", "Few-Shot", "Chain-of-Thought"]:
        p, r, latency, tokens, quality = run(
            compare_task, s,
            "Example: explain a slow function -> profile, find bottleneck, optimize."
        )
        rows.append({
            "Strategy": s,
            "Quality": quality,
            "Latency": round(latency, 4),
            "Tokens": tokens
        })
    st.dataframe(rows, use_container_width=True)

st.caption("Validate generated output. Never expose API keys or sensitive data.")
```

8. Modern Tools / Test Plan

Tools: Python, Streamlit, approved LLM API or simulator, browser interface, and tables/graphs for evaluation.

Test Case 1 – Code generation: generate an email-validation function and verify that prompt and response are displayed.

Test Case 2 – Debugging: provide a program with an undefined variable or boundary error and verify that the fault and correction are explained.

Test Case 3 – Documentation: provide a password-reset requirement and verify that purpose, inputs, process, outputs, constraints, and tests are generated.

Comparison test: run the same engineering task with all three strategies and record quality, latency, and estimated token usage.

9. Results and Validation

Representative expected evaluation table:

Zero-Shot: quality 80/100; latency and tokens measured at runtime.

Few-Shot: quality 85/100; latency and tokens measured at runtime.

Chain-of-Thought: quality 88/100; latency and tokens measured at runtime.

These quality values are illustrative; the final report should replace them with actual execution measurements.

Validation succeeds when all modules accept valid input, show their prompt, return a response, and handle empty input without crashing.

10. Analysis, Engineering Decision, Broader Considerations, Conclusion and Reflection

Zero-shot is simple and compact; few-shot is useful when output format or style must be demonstrated; chain-of-thought-style instructions are useful for complex systematic analysis but may increase prompt length and latency. Therefore, the strategy should be selected according to task complexity rather than using one strategy for every task.

Responsible AI is addressed through prompt transparency, output verification, privacy-aware input handling, secret protection, and explicit communication of model limitations. The lightweight architecture also reduces infrastructure and resource requirements.

Conclusion: PromptCraft integrates prompt engineering, LLM task modules, evaluation, Streamlit implementation, testing, and responsible-AI practices into one unified engineering assistant and satisfies the functional and technical direction of Section D.

Reflection: The project shows that prompting can be treated as an engineering artifact that can be designed, tested, compared, and improved using measurable criteria. Further work could connect a real LLM API, add persistent experiment logs, and evaluate larger engineering datasets.

11. Output Screenshots / Evidence of Implementation

The following representative output screenshots are included as evidence for the assessment requirements. They cover the Prompt Engineering Workbench, engineering task modules, and Prompt Strategy Evaluation. The metrics shown are demonstration values and should be replaced by actual runtime screenshots if the faculty requires execution evidence.

PromptCraft

Prompt Engineering Workbench — Zero-Shot

```
Engineering Task:
Generate Python code to validate an email address.

Prompt Strategy: Zero-Shot

Constructed Prompt:
You are an engineering assistant.
Task: Generate Python code to validate an email address.
Give a clear solution.
```

```
LLM Response:
Engineering response: identify requirements,
design, implementation, edge cases and testing.

Quality: 82/100
Latency: 0.0012 s
Estimated Tokens: 28
```

Streamlit output screenshot — demonstration

PromptCraft

Engineering Task Modules — Debugging

```
Module: Automated Debugging Assistance

Input:
NameError: variable 'total' is not defined

Module Prompt:
Debug this problem and explain the correction.
```

```
Module Output:
1. Locate the undefined variable.
2. Check initialization.
3. Replace/use the correct variable.
4. Retest normal and edge cases.

Status: Valid response
```

Streamlit output screenshot — demonstration

PromptCraft

Engineering Task Modules — Documentation

```
Requirement:
Users shall reset a forgotten password using
their registered email address.

Generated Documentation:
Purpose: Secure password recovery
Input: Registered email
Process: Validate email → issue reset link
Output: Password reset confirmation
```

```
Validation:
✓ Requirement captured
✓ Inputs identified
✓ Process described
✓ Output defined
✓ Constraints documented
```

Streamlit output screenshot — demonstration

.

PromptCraft

Prompt Strategy Evaluation & Comparison

Strategy	Quality	Latency (s)	Tokens
Zero-Shot	80/100	0.0012	28
Few-Shot	85/100	0.0016	43
Chain-of-Thought	88/100	0.0019	39

Streamlit output screenshot — demonstration

.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course / domain knowledge (Prompt Engineering, LLM Integration, Evaluation)	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques (Python, LLM APIs, Prompt Engineering Frameworks)	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	**100**

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO1/CO2	PO1, PO2	L3	10
Application – Prompt Engineering Techniques	CO1/CO2	PO1, PO2, PO3	L3/L4	8
Application – LLM Task Module Integration	CO1	PO1, PO2, PO3	L3/L4	6
Application – Evaluation & Comparison of Prompts	CO2	PO1, PO2, PO3	L3/L4	6
Solution / Design – Integrated PromptCraft	CO1–CO2	PO2, PO3, PO5	L4/L5/L6	20

Modern tool usage (Python, LLM APIs, Prompt Engineering)	CO1–CO2	PO5	L3	10
Validation of results	CO1–CO2	PO4	L4/L5	15
Analysis & justification / trade-offs	CO1–CO2	PO2, PO4	L4/L5	15
Broader considerations & documentation / reflection	CO1–CO2	PO6, PO12	L3/L4	10

Note: PO numbers indicated are illustrative; faculty shall verify against the current CSA08 CO–PO matrix and map only the POs genuinely assessed.

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60 – 69%
Level 1	50 – 59%
Level 0	< 50%

Target CO Attainment: _____ Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____